Advanced Port Scanner

Cybersecurity Technical Report

April 8, 2026

Security Analysis • Network Assessment • Python Tooling

Contents

1 Executive Summary 2

2 Objective 2

3 Tools Used 2

4 Tool Interface (GUI) 3

5 Methodology 3

6 Results 4

7 Complete Python Code 4

8 Conclusion 8

Executive Summary

This project presents the design and implementation of an advanced port scanner developed using Python. The tool is capable of identifying open ports, detecting running services, and providing insights into the target system. The primary objective of this project is to simulate real-world reconnaissance techniques used in cybersecurity assessments and penetration testing.

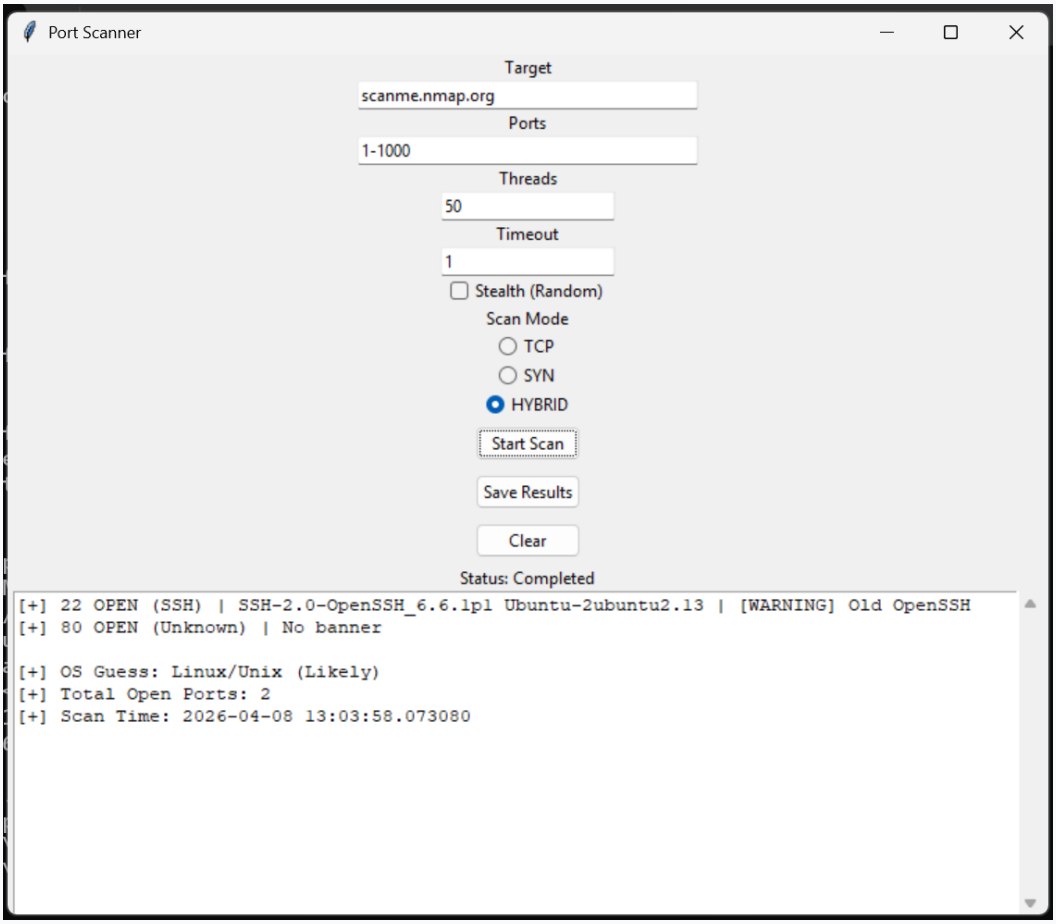
Objective

The main objective of this project is to develop a high-performance port scanning tool that can efficiently identify open ports and services on a target system. Additionally, the project aims to enhance practical knowledge in networking, cybersecurity concepts, and Python programming.

Tools Used

- Python 3 for core development
- Socket Programming for TCP connections
- Tkinter for graphical user interface (GUI)
- Scapy for advanced packet crafting (SYN scanning)
- Multithreading for faster and efficient scanning

Tool Interface (GUI)



The graphical interface allows users to input target IP addresses, define port ranges, select scan types, and view scan results in real time. This improves usability and provides a user-friendly experience.

Methodology

- TCP Connect Scan to establish full connections
- SYN Scan for stealth scanning
- Hybrid Scan combining multiple techniques
- Banner Grabbing for service identification

Results

The scan identified multiple open ports and running services on the target system:

- Port 22 — SSH (Open)
- Port 80 — HTTP (Open)

System Insight: The detected services indicate that the system is likely running a Linux/Unix-based operating system.

Complete Python Code

```python
import socket
import threading
from queue import Queue
import random
import tkinter as tk
from tkinter import ttk, scrolledtext
import datetime

# SYN scan requires scapy
from scapy.all import IP, TCP, sr1


# ================= CORE ================= #

def syn_scan(target, port, timeout):
    try:
        pkt = IP(dst=target)/TCP(dport=port, flags="S")
        response = sr1(pkt, timeout=timeout, verbose=0)

        if response is None:
            return "Filtered"

        elif response.haslayer(TCP):
            flags = response.getlayer(TCP).flags
            if flags == 0x12:
                return "Open"
            elif flags == 0x14:
                return "Closed"

        return "Filtered"
    except:
        return "Error"


def grab_banner(target, port, timeout):
    try:
        sock = socket.socket()
        sock.settimeout(timeout)
        sock.connect((target, port))

        if port in [80, 8080]:
            sock.send(b"GET␣/␣HTTP/1.1\r\nHost:␣test\r\n\r\n")

        banner = sock.recv(1024).decode(errors="ignore")

```

```python
        if "Server:" in banner:
            for line in banner.split("\n"):
                if "Server:" in line:
                    return line.strip()

        return banner.split("\n")[0].strip()
    except:
        return "No banner"


def detect_service(port, banner):
    b = banner.lower()

    if "ssh" in b:
        return "SSH"
    elif "apache" in b:
        return "Apache HTTP"
    elif "nginx" in b:
        return "Nginx HTTP"
    elif "http" in b:
        return "HTTP"
    elif "ftp" in b:
        return "FTP"
    elif port == 443:
        return "HTTPS"
    elif port == 3389:
        return "RDP"
    elif port == 445:
        return "SMB"
    else:
        return "Unknown"


def check_vuln(banner):
    b = banner.lower()

    if "apache/2.4.7" in b:
        return "[VULNERABLE] CVE-2017-3169"
    elif "openssh_6" in b:
        return "[WARNING] Old OpenSSH"
    return ""


def guess_os(open_ports, banners):
    combined = " ".join(banners).lower()

    if "windows" in combined or 3389 in open_ports:
        return "Windows (Likely)"
    elif "ubuntu" in combined or "linux" in combined or 22 in open_ports:
        return "Linux/Unix (Likely)"
    else:
        return "Unknown"


# ================= GUI ================= #

def launch_gui():

    def start_scan():
        target = target_entry.get()
        ports_input = ports_entry.get()
```

```
        threads = int(threads_entry.get())
        timeout = float(timeout_entry.get())
        mode = scan_mode.get()

        output_box.delete(1.0, tk.END)
        progress_label.config(text="Status:␣Scanning...")

        try:
            target_ip = socket.gethostbyname(target)
        except:
            output_box.insert(tk.END, "Invalid␣target\n")
            return

        # Parse ports
        if "-" in ports_input:
            start, end = map(int, ports_input.split("-"))
            ports = list(range(start, end + 1))
        else:
            ports = list(map(int, ports_input.split(",")))

        if random_var.get():
            random.shuffle(ports)

        queue = Queue()
        for port in ports:
            queue.put(port)

        open_ports = []
        banners = []

        def worker():
            while not queue.empty():
                port = queue.get()

                state = "Unknown"
                banner = ""

                try:
                    # TCP Mode
                    if mode == "TCP":
                        sock = socket.socket()
                        sock.settimeout(timeout)
                        result = sock.connect_ex((target_ip, port))

                        if result == 0:
                            state = "Open"
                            banner = grab_banner(target_ip, port, timeout)
                        else:
                            state = "Closed"

                        sock.close()

                    # SYN Mode
                    elif mode == "SYN":
                        state = syn_scan(target_ip, port, timeout)
                        if state == "Open":
                            banner = "N/A␣(SYN)"

                    # HYBRID Mode
                    elif mode == "HYBRID":
                        state = syn_scan(target_ip, port, timeout)
```

```
                        if state == "Open":
                            banner = grab_banner(target_ip, port, timeout)

                    # Handle open ports
                    if state == "Open":
                        service = detect_service(port, banner)
                        vuln = check_vuln(banner)

                        open_ports.append(port)
                        banners.append(banner)

                        msg = f"[+] {port} OPEN ({service}) | {banner}"
                        if vuln:
                            msg += f" | {vuln}"

                        msg += "\n"

                        output_box.insert(tk.END, msg)
                        output_box.see(tk.END)

                except:
                    pass

                queue.task_done()

        # Start threads
        for _ in range(threads):
            threading.Thread(target=worker, daemon=True).start()

        def finish_scan():
            if not queue.unfinished_tasks:
                os_guess = guess_os(open_ports, banners)

                output_box.insert(tk.END, f"\n[+] OS Guess: {os_guess}\n")
                output_box.insert(tk.END, f"[+] Total Open Ports: {len(open_ports)}\n")

                now = datetime.datetime.now()
                output_box.insert(tk.END, f"[+] Scan Time: {now}\n")

                progress_label.config(text="Status: Completed")
            else:
                root.after(500, finish_scan)

        finish_scan()

    def save_results():
        with open("scan_results.txt", "w") as f:
            f.write(output_box.get(1.0, tk.END))

    def clear_output():
        output_box.delete(1.0, tk.END)

    # GUI Layout
    root = tk.Tk()
    root.title("Port Scanner")
    root.geometry("750x620")

    ttk.Label(root, text="Target").pack()
    target_entry = ttk.Entry(root, width=40)
    target_entry.pack()
```

```python

    ttk.Label(root, text="Ports").pack()
    ports_entry = ttk.Entry(root, width=40)
    ports_entry.pack()

    ttk.Label(root, text="Threads").pack()
    threads_entry = ttk.Entry(root)
    threads_entry.insert(0, "50")
    threads_entry.pack()

    ttk.Label(root, text="Timeout").pack()
    timeout_entry = ttk.Entry(root)
    timeout_entry.insert(0, "1")
    timeout_entry.pack()

    random_var = tk.BooleanVar()
    ttk.Checkbutton(root, text="Stealth (Random)", variable=random_var).pack()

    scan_mode = tk.StringVar(value="TCP")

    ttk.Label(root, text="Scan Mode").pack()
    ttk.Radiobutton(root, text="TCP", variable=scan_mode, value="TCP").pack()
    ttk.Radiobutton(root, text="SYN", variable=scan_mode, value="SYN").pack()
    ttk.Radiobutton(root, text="HYBRID", variable=scan_mode, value="HYBRID").pack()

    ttk.Button(root, text="Start Scan", command=start_scan).pack(pady=5)
    ttk.Button(root, text="Save Results", command=save_results).pack(pady=5)
    ttk.Button(root, text="Clear", command=clear_output).pack(pady=5)

    progress_label = ttk.Label(root, text="Status: Idle")
    progress_label.pack()

    output_box = scrolledtext.ScrolledText(root, width=90, height=25)
    output_box.pack()

    root.mainloop()


if __name__ == "__main__":
    launch_gui()
```

Conclusion

This project successfully demonstrates the implementation of a practical port scanning tool using Python. It highlights essential cybersecurity concepts such as reconnaissance, service detection, and attack surface identification. The project also strengthens understanding of networking protocols and real-world security assessment techniques.

Contact

Shameel C

Cybersecurity Engineer | SIEM · VA · Network Security

shameelsafar@gmail.com

+91 99467 17639 • Kerala, India

LinkedIn | GitHub | Portfolio